Name : P. Yaswanth

Reg No :192524167

Experiment: Missionaries and Cannibals Problem

Aim

To solve the Missionaries and Cannibals Problem using Python by applying the **State Space Search (Breadth-First Search)** algorithm.

Objectives

- To understand the Missionaries and Cannibals problem.
- To apply the State Space Search (BFS) algorithm.
- To find a safe sequence of moves to transport all missionaries and cannibals across the river.
- To implement the solution using Python.
- To analyze the effectiveness of the BFS algorithm in solving AI search problems.

Algorithm

1. Represent each state as **(Missionaries Left, Cannibals Left, Boat Side)**.
2. Start with the initial state **(3, 3, 1)** where the boat is on the left bank.
3. Generate all valid successor states by moving one or two people across the river.
4. Reject invalid states where cannibals outnumber missionaries on either bank.
5. Mark visited states to avoid repetition.
6. Continue searching until the goal state **(0, 0, 0)** is reached.
7. Display the sequence of states leading to the solution.

Python Program

```
from collections import deque


# Check whether a state is valid

def is_valid(m_left, c_left):

  m_right = 3 - m_left
```

```
    c_right = 3 - c_left

    if m_left < 0 or c_left < 0 or m_left > 3 or c_left > 3:
        return False

    if (m_left > 0 and m_left < c_left):
        return False

    if (m_right > 0 and m_right < c_right):
        return False

    return True

def bfs():
    start = (3, 3, 1)   # (Missionaries Left, Cannibals Left, Boat Left=1)
    goal = (0, 0, 0)

    queue = deque([(start, [])])
    visited = set()

    while queue:
        state, path = queue.popleft()

        if state in visited:
            continue
```

```
        visited.add(state)
        path = path + [state]

        if state == goal:
            print("Solution Path:")
            for step in path:
                print(step)
            return

        m_left, c_left, boat = state

        moves = [(2,0), (0,2), (1,1), (1,0), (0,1)]

        for m, c in moves:
            if boat == 1:
                new_state = (m_left - m, c_left - c, 0)
            else:
                new_state = (m_left + m, c_left + c, 1)

            if is_valid(new_state[0], new_state[1]):
                queue.append((new_state, path))

    print("No solution found.")

bfs()
```

Sample Output

Solution Path:

(3, 3, 1)

(3, 1, 0)

(3, 2, 1)

(3, 0, 0)

(3, 1, 1)

(1, 1, 0)

(2, 2, 1)

(0, 2, 0)

(0, 3, 1)

(0, 1, 0)

(1, 1, 1)

(0, 0, 0)

Out Put

```python
from collections import deque

# Check whether a state is valid
def is_valid(m_left, c_left):
    m_right = 3 - m_left
    c_right = 3 - c_left

    if m_left < 0 or c_left < 0 or m_left > 3 or c_left > 3:
        return False

    if (m_left > 0 and m_left < c_left):
        return False

    if (m_right > 0 and m_right < c_right):
        return False

    return True

def bfs():
    start = (3, 3, 1)   # (Missionaries Left, Cannibals Left, Boat Left=1)
    goal = (0, 0, 0)

    queue = deque([(start, [])])
    visited = set()

    while queue:
```

```
Solution Path:
(3, 3, 1)
(3, 1, 0)
(3, 2, 1)
(3, 0, 0)
(3, 1, 1)
(1, 1, 0)
(2, 2, 1)
(0, 2, 0)
(0, 3, 1)
(0, 1, 0)
(1, 1, 1)
(0, 0, 0)

=== Code Execution Successful ===
```

Result

The Missionaries and Cannibals Problem was successfully solved using the **State Space Search (Breadth-First Search)** algorithm. The program found a valid sequence of moves that safely transported all missionaries and cannibals across the river.

Conclusion

The experiment demonstrated the use of the **Breadth-First Search (BFS)** algorithm to solve the Missionaries and Cannibals problem. By exploring valid states while avoiding unsafe configurations and repeated states, the program successfully reached the goal state. This experiment illustrates the application of state-space search techniques in Artificial Intelligence.